

#1: CS1 Review

Goals for today's class: CS1 Review

- Functions
 - ▶ parameters/arguments
 - ▶ return statements
- Conditionals
 - ▶ boolean expressions
 - ▶ if-else statements
 - ▶ nested/chained conditionals
- Loops
 - ▶ for loops with `range(·)`
 - ▶ while loops
- Lists
 - ▶ creating a list
 - ▶ accessing a list element
- Loop patterns
 - ▶ min/max, counter, quantifier (“for all” / “for some”)

Outline

- 1 **Functions**
- 2 Conditionals
- 3 Loops
- 4 Lists
- 5 Loop Patterns

- 대부분의 경우에, **return** value가 있는 함수를 만들게 됨

Example (Function with return value)

```
def distance(x1,y1,x2,y2):  
    u, v = (x2-x1)**2, (y2-y1)**2  
    return math.sqrt(u+v)
```

```
print(distance(0,0,3,4))    # 화면에 5를 뿌림
```

- 가끔씩은 **return** value가 **없**는 함수를 만들 필요도 있음
 - ▶ 이 경우에는 함수 내부에서 **print**를 하게 됨
 - ▶ 자동으로 **None**이 return됨

Example (Function without return value)

```
def distance(x1,y1,x2,y2):  
    u, v = (x2-x1)**2, (y2-y1)**2  
    print(math.sqrt(u+v))    # 여기서 화면에 5를 뿌림
```

```
distance(0,0,3,4)           # return value가 없음  
print(distance(0,0,3,4))    # 화면에 None를 출력
```

Outline

- 1 Functions
- 2 Conditionals**
- 3 Loops
- 4 Lists
- 5 Loop Patterns

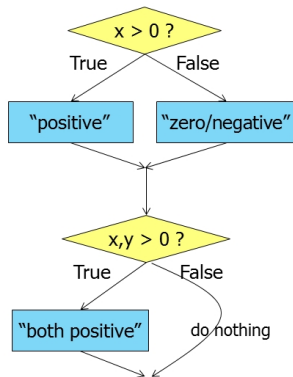
if-else statements

Syntax

```
if boolean_expression :           # (either True or False)
    statements
else:                               # else block may be omitted
    statements
```

```
def f(x,y):
    if x > 0:
        print("positive")
    else:
        print("zero/negative")

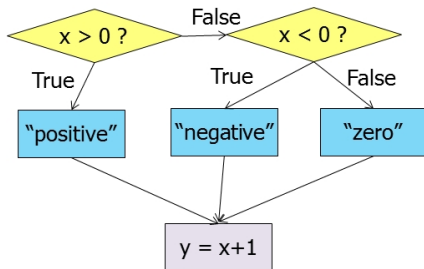
    if x > 0 and y > 0:
        print("both positive")
```



Chained if-else statements

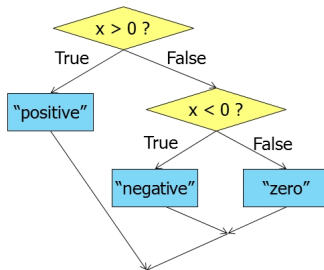
```
if b1 :  
    statements  
elif b2 :  
    statements          # executed if b1 is False and b2 is True  
elif b3 :  
    statements          # executed if b1 ∨ b2 is False and b2 is True  
...  
else:  
    statements          # else block may be omitted
```

```
def f(x):  
    if x > 0:  
        print("positive")  
    elif x < 0:  
        print("negative")  
    else:  
        print("zero")  
  
    y = x+1 ...
```



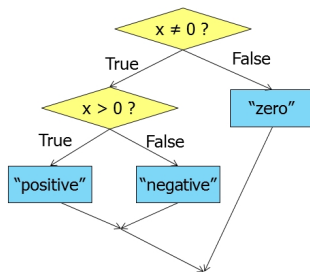
Nested if-else statements

```
if x > 0:
    print("positive")
else:
    if x < 0:
        print("negative")
    else:
        print("zero")
```



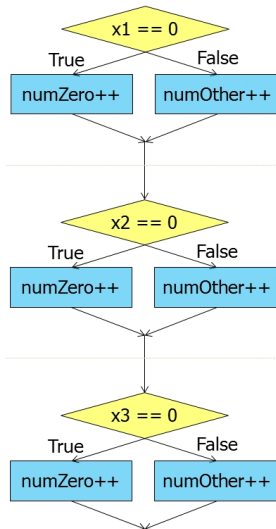
VS.

```
if x != 0:
    if x > 0:
        print("positive")
    else:
        print("negative")
else:
    print("zero")
```



Example: Counting the number of 0's

```
def f(x1, x2, x3):  
    numZero = 0  
    numOther = 0  
  
    if x1 == 0:  
        numZero = numZero+1  
    else:  
        numOther = numOther+1  
  
    if x2 == 0:  
        numZero = numZero+1  
    else:  
        numOther = numOther+1  
  
    if x3 == 0:  
        numZero = numZero+1  
    else:  
        numOther = numOther+1  
  
    print(numZero, numOther)
```



Making a problem simpler by using “pruning” (가지치기)

```
def absoluteValue(x):  
    if x < 0:  
        return -x  
    else:  
        return x
```

 $\equiv$

```
def absoluteValue(x):  
    if x < 0:  
        return -x  
    return x
```

complicated nested structure

```
if b1:  
    return v1  
else:  
    if b2:  
        return v2  
    else:  
        if b3:  
            return v3  
        else:  
            return v4
```

 $\Rightarrow$

simpler one by pruning

```
if b1:  
    return v1  
  
if b2:  
    return v2  
  
if b3:  
    return v3  
  
return v4
```

Defining/Using Boolean Functions

- Boolean functions return **boolean** values (**True/False**)
- Useful in **hiding complicated tests** inside `if ....`
 - ▶ What if we do not define/use `onePositive()`?
- Faced with **yes/no questions**, consider boolean functions.

```
def onePositive(x,y,z):  
    if x > 0 and y <= 0 and z <= 0:  
        return True  
    if x <= 0 and y > 0 and z <= 0:  
        return True  
    if x <= 0 and y <= 0 and z > 0:  
        return True  
    return False  
  
if onePositive(x,y,z):  
    print("Exactly one number is positive.")
```

Tips: Return boolean expressions if possible

```
def lessThan(x,y):  
    if x < y:  
        return True  
    else:  
        return False
```



```
def lessThan(x,y):  
    return x < y  
    # x < y is True/False
```

```
if boolean_expr :  
    return True  
else:  
    return False
```



```
return boolean_expr
```

```
if boolean_expr :  
    return False  
else:  
    return True
```



```
return not boolean_expr
```

Tips: Remove == True and Replace == False by not

- `b == True` $\iff$ `b`
- `b == False` $\iff$ `not b`

```
def singleDigit(x):  
    return (0 <= x < 10)  
  
...  
if singleDigit(a) == True and singleDigit(b) == False:  
    ...
```



```
if singleDigit(a) and not singleDigit(b):
```

Common mistake: use `=` instead of `==`

- `if singleDigit(a) = True: ...`
- `if singleDigit(a) = False: ...`

Outline

1 Functions

2 Conditionals

3 Loops

4 Lists

5 Loop Patterns

for Loops with range

```
for k in range(n):  
    print(k)                # 0 1 ... n-1  
  
for k in range(1, n+1):  
    print(k)                # 1 2 ... n  
  
for k in range(10, 21, 2):  
    print(k)                # 10 12 14 16 18 20  
  
for k in range(n, 0, -1):  
    print(k)                # n n-1 ... 2 1
```

- `range(m, n) ≡ range(m, n, 1)`
- `range(n) ≡ range(0, n) ≡ range(0, n, 1)`

Example: Computing sum and factorial

Computing $1 + 2 + \dots + n$

```
def sum(n):  
    x = 0 # 덧셈에 대한 항등원은 0  
  
    for i in range(1,n+1):    # i: 1,2,...n  
        x += i  
  
    return x
```

Computing $1 \cdot 2 \cdot \dots \cdot n$

```
def factorial(n):  
    x = 1 # 곱셈에 대한 항등원은 1  
  
    for i in range(1,n+1):    # i: 1,2,...n  
        x *= i  
  
    return x
```


Example: Printing multiplication table

```
for i in range(1, 10):  
    for j in range(1, 10):  
        print(i*j, end=" ")  
    print()
```

1	2	3	4	5	6	7	8	9
2	4	6	8	10	12	14	16	18
3	6	9	12	15	18	21	24	27
4	8	12	16	20	24	28	32	36
5	10	15	20	25	30	35	40	45
6	12	18	24	30	36	42	48	54
7	14	21	28	35	42	49	56	63
8	16	24	32	40	48	56	64	72
9	18	27	36	45	54	63	72	81

```
for i in range(??):  
    for j in range(??):  
        print(i*j, end=" ")  
    print()
```

1	2	3	4	5	6	7	8	9
3	6	9	12	15	18	21	24	27
5	10	15	20	25	30	35	40	45
7	14	21	28	35	42	49	56	63
9	18	27	36	45	54	63	72	81

```
for i in range(??):  
    for j in range(??):  
        print(i*j, end=" ")  
    print()
```

1								
3	6	9						
5	10	15	20	25				
7	14	21	28	35	42	49		
9	18	27	36	45	54	63	72	81

while Loops

Syntax of while loops

```
while boolean_expression :  
    statements
```

Repeat *statements* while *boolean_expression* is true

Counting the number of digits

```
def countDigits(n):  
    counter = 0  
  
    while n > 0:  
        counter += 1  
        n = n//10  
  
    return counter
```

```
countDigits(n = 713)  
    while 713 > 0:  
        counter = 1 / n = 71  
    while 71 > 0:  
        counter = 2 / n = 7  
    while 7 > 0:  
        counter = 3 / n = 0  
    while 0 > 0:  
        return counter = 3
```

Outline

- 1 Functions
- 2 Conditionals
- 3 Loops
- 4 Lists**
- 5 Loop Patterns

Creating/Using Lists

초기값을 주면서 생성

```
number = [ 2, 5, 8, 11, 14 ]
```

초기값을 나중에 채워넣는 생성 방식

```
number = [None] * 5      # [None, None, None, None, None]  
number[0] = 2 ...
```

len(·)과 [i]로 list 원소 쓰기/읽기

```
for i in range(len(number)): # list length: 5  
    number[i] = 3*i+2        # i: index
```

Index는 0에서 시작 len(·)-1에서 끝남! (1부터 시작 아님)

- 그러면 range(·)하고도 잘 맞아서 for 루프에서 편리
- [-1]은 [len(·)-1]과 같음 (마지막 원소)

List가 함수의 입력/출력으로 쓰이는 예

```
def func(b):  
    a = [None] * len(b)  
  
    for i in range(len(a)):  
        a[i] = b[i] + 1  
  
    return a  
  
num = [ 2, 4, 3, 1, 7, 2, 5, 6 ]  
c = func(num)  
  
print(c)  # [3, 5, 4, 2, 8, 3, 6, 7]
```

Outline

- 1 Functions
- 2 Conditionals
- 3 Loops
- 4 Lists
- 5 Loop Patterns**

Basic Patterns of for Loops

Given a list numbers,

1: Maximum/Minimum

- Find the **maximum** value among numbers
- Find the **index** i that **maximizes** numbers[i]

2: Counter

- **Count** the **number** of primes in numbers.
- **Count** the **number** of composites in points.

3: Quantifier (“for some” / “for all”)

- Determine if numbers[i] > 0 **for some** i .
- Determine if numbers[i] > 0 **for all** i .
- Determine if numbers[i] is prime **for some** i .

대부분의 for 루프는 위 패턴들의 조합으로 만들어질 수 있음

1: Maximum/Minimum Pattern

What is the minimum value?

```
def findMin(numbers):  
    min = numbers[0]  
    for i in range(len(numbers)):  
        if numbers[i] < min:  
            min = numbers[i]  
    return min
```

최소/최대 “값” 이 아니고 그 “위치” 를 구할 필요도 있음

What is the “index” of the minimum value?

```
def findMinIndex(numbers):  
    minIndex = 0  
    for i in range(len(numbers)):  
        if numbers[i] < numbers[minIndex]:  
            minIndex = i  
    return minIndex
```


2: Counter Pattern

```
counter = 0
for i in range(len(numbers)):
    if condition(numbers[i]) is True:
        counter += 1
```

Count the number of elements that satisfy some condition

Counting the number of primes

```
def countPrime(numbers):
    counter = 0
    for i in range(len(numbers)):
        if isPrime(numbers[i]): # boolean function
            counter += 1
    return counter
```

Boolean expression 자리의 코드만 바꿔주면 됨

3: Quantifier (“for some” / “for all”) Pattern

- `numbers[i] > 0` **for some** `i` ?

```
def somePositive(numbers):  
    for i in range(len(numbers)):  
        if numbers[i] > 0:  
            return True  
    return False
```

- `numbers[i] > 0` **for all** `i` ?
 - ▶ `not (not (numbers[i] > 0) for some i)`

```
def allPositive(numbers):  
    for i in range(len(numbers)):  
        if not(numbers[i] > 0):      # numbers[i] <= 0  
            return False  
    return True
```

3: Quantifier (“for some” / “for all”) Pattern

De Morgan's laws: $\forall x, p(x) \equiv \neg (\exists x, \neg p(x))$

$\exists x \in X, p(x)$

```
for x ∈ X:  
    if p(x):  
        return True  
return False
```

$\forall x \in X, p(x)$

```
for x ∈ X:  
    if not p(x):  
        return (not True)    # False  
return (not False)         # True
```

이번 학기에 자주 쓰이는 함수: 소수 판별

“p is prime” $\equiv$ “p%i != 0 for all i”

```
def isPrime(p):    # "for all" pattern  
    if p < 2: return False  
    for i in range(2, p//2+1):  
        if not (p%i != 0):    # p%i == 0  
            return False  
    return True
```